

Principios del Hardware de E/S

Dispositivos de E/S

Se dividen en dos categorías:

- **Dispositivos de bloque:** almacena información en bloques de tamaño fijo, cada uno con su propia dirección. Todas las transferencias se realizan en unidades de uno o más bloques completos. Es posible leer o escribir cada bloque de manera independiente de los demás. HDD, CD-ROMs, USBs.
- **Dispositivos de carácter:** envía o acepta un flujo de caracteres, sin importar la estructura del bloque. No es direccionable y no tiene ninguna operación de búsqueda. Impresoras, interfaces de red, ratones, teclados.

Controladores de dispositivos

Las unidades de E/S consisten en un componente mecánico (dispositivo en sí) y un componente electrónico (controlador de dispositivo, es **HARDWARE**).

El trabajo del controlador es convertir el flujo de bits serial en un bloque de bytes y realizar cualquier corrección de errores necesaria. Luego copia a la RAM los datos después de haber hecho las verificaciones correspondientes.

E/S por asignación de memoria

¿Cómo se comunica la CPU con los registros de control y los **buffers** de datos de los dispositivos?

Existen dos alternativas:

1. Primer método, a cada registro de control se le asigna un número de puerto de E/S, un entero de 8 o 16 bits. El conjunto de todos los puertos de E/S forma el **espacio de puertos de E/S** y está protegido de manera que los programas de usuario ordinarios no puedan utilizarlo (solo el SO puede). Figura (a).
2. Segundo método, es asignar todos los registros de control al espacio de memoria, como se muestra en la figura (b). A cada registro de control se le asigna una dirección de memoria única a la cual no hay memoria asignada. Este sistema se conoce como **E/S con asignación de memoria** (*mapped-memory*).
3. En la figura (c) se muestra un esquema híbrido, con **buffers** de datos de E/S por asignación de memoria y puertos de E/S separados para los registros de control.

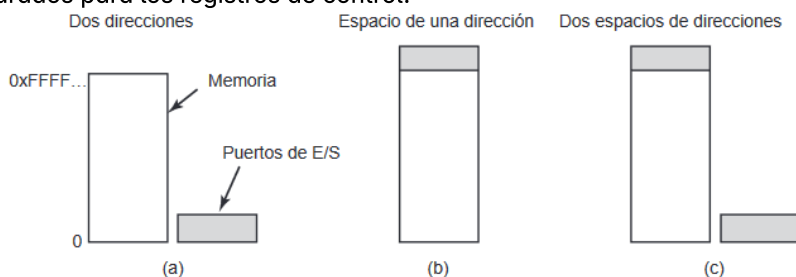


Figura 5-2. (a) Espacio separado de E/S y memoria. (b) E/S por asignación de memoria. (c) Híbrido.

* ¿Cómo funcionan estos esquemas?

En todos los casos, cuando la CPU desea leer una palabra ya sea de la memoria o de un puerto de E/S, coloca la dirección que necesita en las líneas de dirección del bus y después impone una señal READ en una línea de control del bus. Se utiliza una segunda línea de señal para indicar si se necesita espacio de E/S o de memoria. Si es espacio de memoria, ésta responde a la petición. Si es espacio de E/S, el dispositivo de E/S responde a la petición. Si sólo hay espacio de memoria [como en la figura 5-2(b)], todos los módulos de memoria y todos los dispositivos de E/S comparan las líneas de dirección con el rango de direcciones a las que dan servicio. Si la dirección está en su rango, responde a la petición. Como ninguna dirección se asigna tanto a la memoria como a un dispositivo de E/S, no hay ambigüedad ni conflicto.

La E/S por asignación de memoria tiene sus desventajas👉

1. El hardware debe ser capaz de deshabilitar la caché en forma selectiva (por página). Esto agrega una complejidad adicional al hardware y al sistema operativo.
2. Si la computadora tiene un solo bus, como en la figura 5-3(a), es simple hacer que todos analicen cada dirección. Sin embargo, las computadoras modernas tienen un bus dedicado a alta velocidad. El problema es que los dispositivos no tienen forma de ver las direcciones de memoria a medida que recorren el bus de memoria, por lo que no tienen manera de responderles.

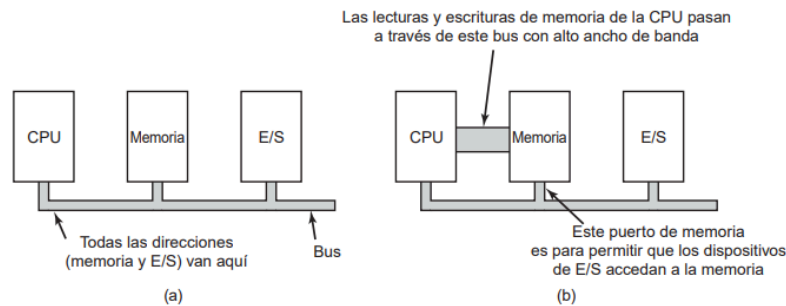


Figura 5-3. (a) Arquitectura con un solo bus. (b) Arquitectura con bus dual.

Acceso directo a memoria (DMA)

El controlador de DMA tiene acceso al bus del sistema de manera independiente de la CPU.

Sin DMA

Sin DMA

el controlador de disco lee el bloque (uno o más sectores) de la unidad en forma serial, bit por bit, hasta que se coloca todo el bloque completo en el búfer interno del controlador. Después calcula la suma de comprobación para verificar que no hayan ocurrido errores de lectura. Después, el controlador produce una interrupción. Cuando el sistema operativo empieza a ejecutarse, puede leer el bloque de disco del búfer del controlador, un byte o una palabra a la vez, mediante la ejecución de un ciclo en el que cada iteración se lee un byte o palabra de un registro de dispositivo controlador y se almacena en la memoria principal.

Con DMA

Con DMA

La CPU programa el controlador de DMA, para lo cual establece sus registros de manera que sepa qué debe transferir y a dónde (paso 1). También emite un comando al controlador de disco para indicarle que debe leer datos del disco en su búfer interno y verificar la suma de comprobación. Cuando hay datos válidos en el búfer del controlador de disco, puede empezar el DMA.

El controlador de DMA inicia la transferencia enviando una petición de lectura al controlador de disco mediante el bus (paso 2). Esta petición de lectura se ve como cualquier otra petición de lectura, por lo que el controlador de disco no sabe ni le importa si vino de la CPU o de un controlador de DMA. Por lo general, la dirección de memoria en la que se va a escribir está en las líneas de dirección del bus, por lo que cuando el controlador de disco obtiene la siguiente palabra de subbúfer interno, sabe dónde escribir. La escritura en memoria es otro ciclo de bus estándar (paso 3). Cuando se completa la escritura, el controlador de disco envía una señal de reconocimiento al controlador de DMA, también a través del bus (paso 4). El controlador de DMA incrementa a continuación la dirección de memoria a utilizar y disminuye la cuenta de bytes. Si la cuenta de bytes es aún mayor que 0, se repiten los pasos del 2 al 4 hasta que la cuenta llega a 0. En ese momento, el controlador de DMA interrumpe la CPU para hacerle saber que la transferencia está completa. Cuando el sistema operativo se inicia, no tiene que copiar el bloque de disco en la memoria; ya se encuentra ahí.

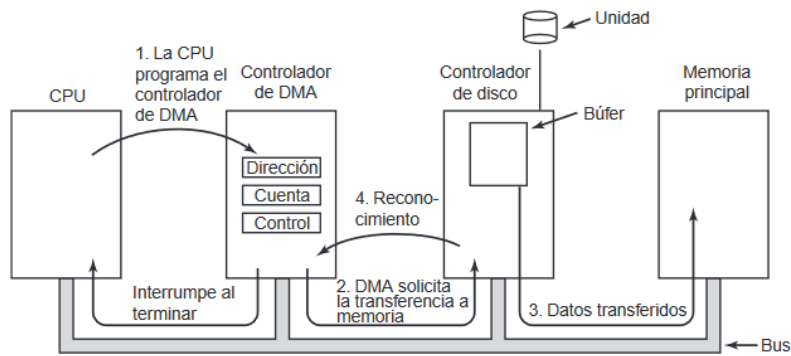


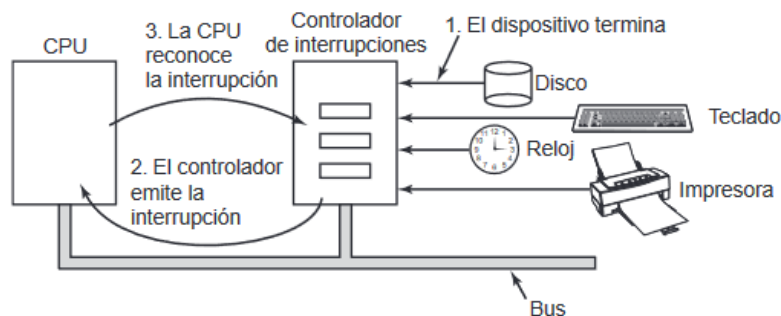
Figura 5-4. Operación de una transferencia de DMA.

En resumen el DMA:

- Tiene acceso al bus,
- Libera CPU de la tarea de I/O. Interrumpe solo una vez (cuando termina),
- Modo Palabra o Bloque:
 - Por palabra cuando DMA se apropia del Bus la CPU debe esperar → Robo de ciclos
 - Por Bloque adquiere el bus, transfiere y libera el bus → Modo ráfaga
- Transfiere directamente a la memoria principal

Interrupciones

Cuando un dispositivo de E/S ha terminado el trabajo que se le asignó, produce una interrupción (suponiendo que el sistema operativo haya habilitado las interrupciones). Para ello, impone una señal en una línea de bus que se le haya asignado. Esta señal es detectada por el chip controlador de interrupciones en la tarjeta principal, que después decide lo que debe hacer.



- Si no hay otras interrupciones pendientes, el controlador de interrupciones procesa la interrupción de inmediato.
- Si hay otra en progreso, o si otro dispositivo ha realizado una petición simultánea en una línea de petición de interrupción de mayor prioridad en el bus, el dispositivo sólo se ignora por el momento.

Interrupciones precisas e imprecisas

Una interrupción que deja al equipo en un estado bien definido se conoce como **interrupción precisa**. Tiene 4 propiedades:

1. El contador del programa (PC) se guarda en un lugar conocido.
2. Todas las instrucciones antes de la instrucción a la que apunta el PC se han ejecutado por completo.
3. Ninguna instrucción más allá de la instrucción a la que apunta el PC se ha ejecutado.
4. Se conoce el estado de ejecución de la instrucción a la que apunta el PC.

Una interrupción que no cumple con estos requerimientos se conoce como **interrupción imprecisa**.

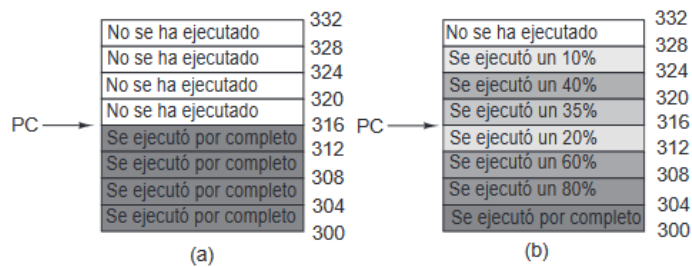


Figura 5-6. (a) Una interrupción precisa. (b) Una interrupción imprecisa.

💡 ¿Para que sirve una interrupción imprecisa?

Fundamentos del Software de E/S

Objetivos del software de E/S

Independencia de dispositivos: escribir programas que puedan acceder a cualquier dispositivo de E/S sin tener que especificar el dispositivo por adelantado. El SO debe resolver el uso de distintos dispositivos, que necesitan distintas ordenes para manejarlos.

Denominación uniforme: El nombre de un archivo o dispositivo simplemente debe ser una cadena o un entero sin depender del dispositivo de ninguna forma.

Manejo de errores: los errores se deben manejar lo más cerca del hardware que sea posible, pero si no es posible el SW debe detectarlo o indicar a la aplicación (disco lleno, file not found)

Transferencias **síncronas** (de bloqueo) vs **asíncronas** (controladas por interrupciones). La mayoría de las operaciones de E/S son asíncronas: se inicia la transferencia y el CPU hace algo más hasta que llega la interrupción. Depende del sistema operativo hacer que las operaciones que en realidad son controladas por interrupciones parezcan de bloqueo para los programas de usuario.

Uso del buffer: a veces los datos que provienen de dispositivo no se pueden almacenar directamente en su destino final, por lo que los datos se deben colocar en un buffer de salida por adelantado para desacoplar la velocidad a la que se llena el buffer, de la velocidad a la que se vacía, de manera que se eviten sub-desbordamientos de búfer.

Dispositivos compartidos y los dispositivos dedicados:

- Los discos son compartidos ya que pueden ser utilizados por varios usuarios a la vez.
- Las cintas son dedicados a un solo usuario hasta que éste termine.

E/S Programada

La CPU hace todo el trabajo.

Ejemplo

El proceso de impresión de una cadena de ocho caracteres en un sistema operativo implica los siguientes pasos, como se muestra en la Figura 5-7:

1. **Preparación de la cadena (Figura 5-7a):** El usuario ensambla la cadena "ABCDEFGH" en un búfer en el espacio de usuario. Luego, realiza una llamada al sistema para obtener acceso a la impresora. Si la impresora está ocupada, el proceso puede fallar o bloquearse hasta que la impresora esté disponible.
2. **Transferencia al espacio de kernel:** Una vez que se obtiene la impresora, el sistema operativo copia la cadena del espacio de usuario al espacio de kernel para gestionarla de manera más eficiente.
3. **Impresión del primer carácter (Figura 5-7b):** El sistema operativo copia el primer carácter ("A") de la cadena al registro de datos de la impresora. El sistema se asegura de que la impresora esté lista para imprimir antes de enviar el carácter, utilizando la técnica de sondeo o espera ocupada. La impresora marca "B" como el siguiente carácter a imprimir.
4. **Impresión de los siguientes caracteres (Figura 5-7c):** El sistema repite el proceso, enviando cada carácter uno por uno a la impresora. En este punto, se ha impreso "AB" y "C" es el siguiente carácter a

enviar.

Este proceso continúa hasta que todos los caracteres de la cadena son impresos. La CPU verifica constantemente si la impresora está lista para aceptar el siguiente carácter, en un ciclo de espera activa.

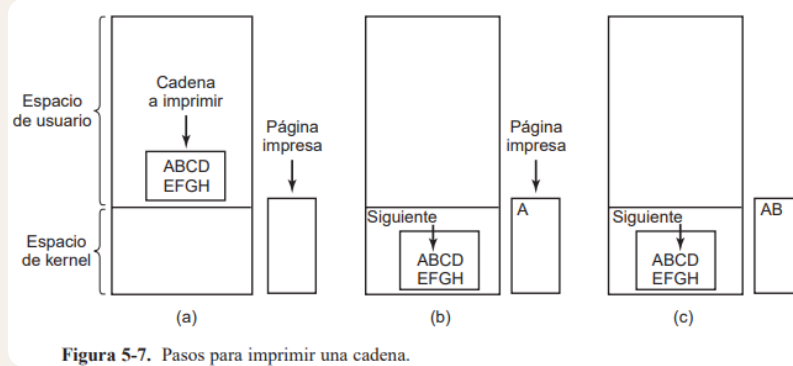


Figura 5-7. Pasos para imprimir una cadena.

E/S controlada por interrupciones

En el caso de la **E/S controlada por interrupciones**, el proceso de impresión en una impresora que imprime cada carácter a medida que llega permite optimizar el uso de la CPU. Si la impresora imprime a 100 caracteres por segundo, cada carácter toma 10 ms para procesarse. Durante este tiempo, la CPU podría estar inactiva, pero en lugar de desperdiciar este tiempo, se utilizan **interrupciones**.

Cuando el sistema realiza una llamada para imprimir, copia el **buffer** al espacio de kernel y envía el primer carácter a la impresora. En lugar de esperar, la CPU permite que otro proceso se ejecute. El proceso de impresión se bloquea hasta que la impresora ha impreso toda la cadena.

Una vez que la impresora está lista para el siguiente carácter, genera una **interrupción** que detiene el proceso en curso, guarda su estado y ejecuta el manejador de interrupciones de la impresora. Este código decide si hay más caracteres por imprimir; si no, desbloquea al proceso que solicitó la impresión. Si aún quedan caracteres, imprime el siguiente y reanuda el proceso interrumpido, continuando desde donde se quedó.

E/S mediante el uso de DMA

Una obvia desventaja de la E/S controlada por interrupciones es que ocurre una interrupción en cada carácter. Una solución es utilizar DMA. La idea es permitir que el controlador de DMA alimente los caracteres a la impresora uno a la vez, sin que la CPU se moleste. En esencia, el DMA es E/S programada, sólo que el controlador de DMA realiza todo el trabajo en vez de la CPU principal.

La gran ganancia con DMA es reducir el número de interrupciones, de una por cada carácter a una por cada **buffer** impreso. Si hay muchos caracteres y las interrupciones son lentas, esto puede ser una gran mejora. Por otra parte, el controlador de DMA es comúnmente más lento que la CPU principal. Si el controlador de DMA no puede controlar el dispositivo a toda su velocidad, o si la CPU por lo general no tiene nada que hacer mientras espera la interrupción de DMA, entonces puede ser mejor utilizar la E/S controlada por interrupción o incluso la E/S programada. De todas formas, la mayor parte del tiempo vale la pena usar DMA.

Capas del Software de E/S

El software de E/S se organiza en 4 capas, cada capa tiene una función e interfaz bien definida por los niveles adyacentes:

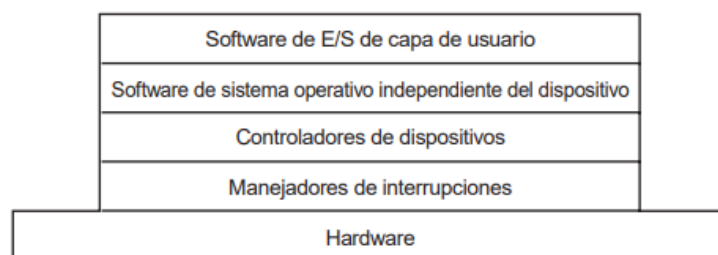


Figura 5-11. Capas del sistema de software de E/S.

Manejadores de interrupciones

El manejador de interrupciones debe ocultarse lo más posible del sistema operativo.

La mejor manera de ocultarlas es hacer que el controlador (driver) que inicia una operación de E/S se bloquee hasta que se haya completado la E/S y ocurra la interrupción. El controlador se puede bloquear a sí mismo realizando una llamada a **down** en un semáforo 🚦, una llamada a **wait** en una variable de condición, una llamada a **receive** en un mensaje o algo similar, por ejemplo.

🔍 ¿Cómo se ejecuta una interrupción?

1. Guardar los registros (incluyendo el PSW) que no han sido guardados por el hardware de la interrupción.
2. Establecer un contexto para el procedimiento de servicio de interrupciones. Para ello tal vez sea necesario establecer el TLB, la MMU y una tabla de páginas.
3. Establecer una pila para el procedimiento de servicio de interrupciones.
4. Reconocer el controlador de interrupciones. Si no hay un controlador de interrupciones centralizado, rehabilitar las interrupciones.
5. Copiar los registros desde donde se guardaron (posiblemente en alguna pila) a la tabla de procesos.
6. Ejecutar el procedimiento de servicio de interrupciones. Éste extraerá información de los registros del controlador de dispositivos que provocó la interrupción.
7. Elegir cuál proceso ejecutar a continuación. Si la interrupción ha ocasionado que cierto proceso de alta prioridad que estaba bloqueado cambie al estado listo, puede elegirse para ejecutarlo en ese momento.
8. Establecer el contexto de la MMU para el proceso que se va a ejecutar a continuación. También puede ser necesario establecer un TLB.
9. Cargar los registros del nuevo proceso, incluyendo su PSW.
10. Empezar a ejecutar el nuevo proceso.

Drivers del dispositivo

Cada dispositivo de E/S conectado a una computadora necesita cierto código específico para controlarlo, este código es el **driver**. Cada sistema operativo necesita sus propios drivers, los fabricantes de dispositivos por lo común los proporcionan para varios SO.

Para poder utilizar el hardware del dispositivo, el driver por lo general **tiene que formar parte del kernel** del sistema operativo, cuando menos en las arquitecturas actuales. En realidad es posible construir controladores que se ejecuten en el espacio de usuario, con llamadas al sistema para leer y escribir en los registros del dispositivo. Este diseño aísla al kernel de los controladores, y a un controlador de otro, eliminando una fuente importante de fallas en el sistema: controladores con errores que interfieren con el kernel de una manera u otra. La comunicación entre el **driver** y el dispositivo se realiza a través del bus.

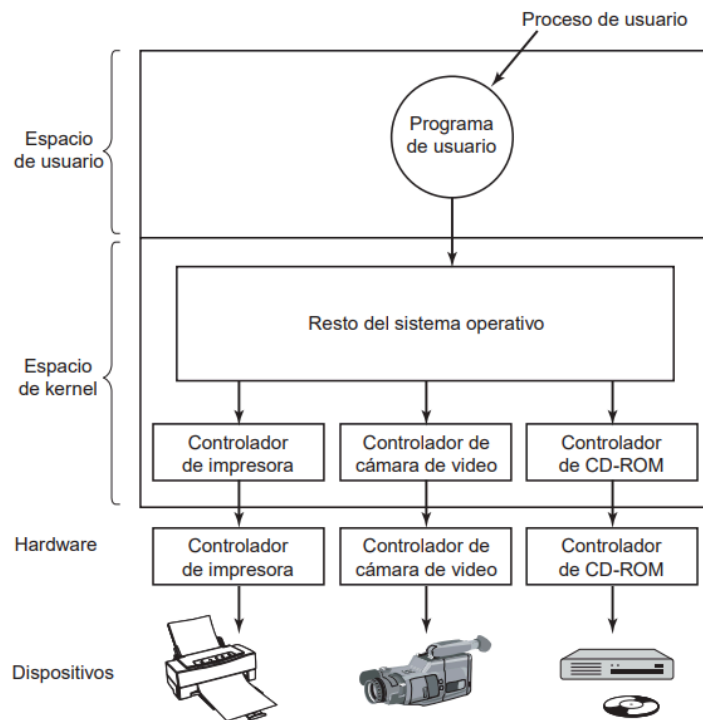


Figura 5-12. Posicionamiento lógico del software controlador de dispositivos. En realidad toda la comunicación entre el software controlador y los controladores de dispositivos pasa a través del bus.

Software de E/S independiente del dispositivo

El límite exacto entre los controladores y el software independiente del dispositivo depende del sistema (y del dispositivo). Las funciones que se realizan comúnmente en el software independiente del dispositivo:

Interfaz uniforme para controladores de dispositivos
Uso de búfer
Reporte de errores
Asignar y liberar dispositivos dedicados
Proporcionar un tamaño de bloque independiente del dispositivo

Figura 5-13. Funciones del software de E/S independiente del dispositivo.

Interfaz uniforme para los controladores de software de dispositivos

Se necesita que todos los dispositivos de E/S y sus controladores se vean más o menos iguales.

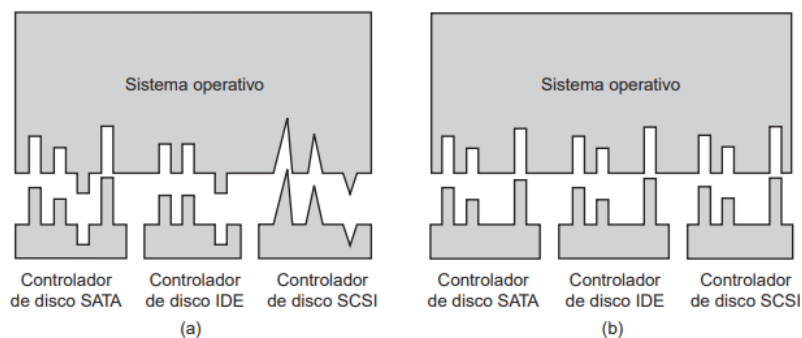


Figura 5-14. (a) Sin una interfaz de controlador estándar. (b) Con una interfaz de controlador estándar.

En la práctica no todos los dispositivos son absolutamente idénticos, pero por lo general hay sólo un pequeño número de tipos de dispositivos, e incluso éstos en general son casi iguales.

✓ Funcionamiento

Para cada clase de dispositivos, como los discos o las impresoras, el sistema operativo define un conjunto de funciones que el controlador debe proporcionar. Para un disco, estas funciones incluyen naturalmente la

lectura y la escritura, pero también encender y apagar la unidad, aplicar formato y otras cosas relacionadas con los discos. A menudo, el controlador contiene una tabla con apuntadores a sí mismo para estas funciones. Cuando se carga el controlador, el sistema operativo registra la dirección de esta tabla de apuntadores a funciones, por lo que cuando necesita llamar a una de las funciones, puede hacer una llamada indirecta a través de esta tabla. Esta tabla de apuntadores a funciones define la interfaz entre el controlador y el resto del sistema operativo. Todos los dispositivos de una clase dada (discos, impresoras, etc.) deben obedecerla.

Otro aspecto de tener una interfaz uniforme es la forma en que se nombran los dispositivos, que debe ser uniforme. En POSIX el nombre de un dispositivo como `/dev/disk0` especifica de manera única el nodo-*i* para un archivo especial, y este nodo-*i* contiene el **número mayor de dispositivo**, que apunta al *driver* (software). El nodo-*i* también contiene el **número menor de dispositivo**, que apunta a la unidad (hardware).

Uso de *buffer*

Tipo de búfer	Características	Problema
Sin búfer	Los caracteres se imprimen a medida que llegan, sin almacenamiento previo.	Ineficiente en cuanto al uso de la CPU, que espera activamente por cada carácter.
Con búfer en usuario	La cadena se almacena en el espacio de usuario antes de transferirla al kernel para imprimir.	El kernel necesita copiar los datos del espacio de usuario, lo que implica más trabajo y tiempo.
Con búfer en kernel	La cadena se almacena en el espacio de kernel para facilitar el acceso del sistema operativo a los datos.	El kernel debe esperar a que la impresora esté lista para cada carácter, introduciendo retardos.
Doble búfer en kernel	Dos búferes se alternan en el kernel: uno se llena mientras el otro se vacía, permitiendo un flujo continuo.	Mayor complejidad en la implementación y uso de memoria adicional para mantener ambos búferes.

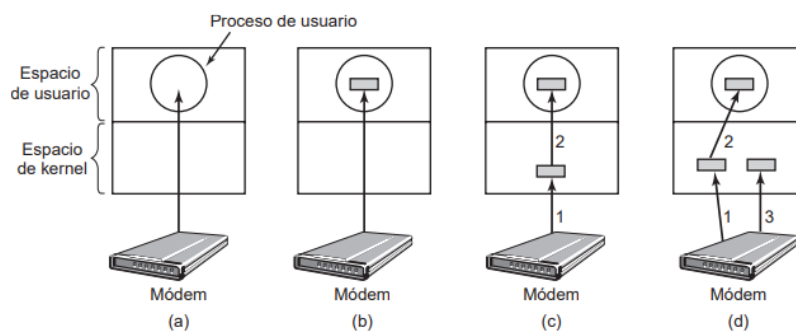


Figura 5-15. (a) Entrada sin búfer. (b) Uso de búfer en espacio de usuario. (c) Uso de búfer en el kernel, seguido de la acción de copiar al espacio de usuario. (d) Uso de doble búfer en el kernel.

Reporte de errores

Los errores son mucho más comunes en el contexto de la E/S que en otros. Cuando ocurren, el sistema operativo debe manejarlos de la mejor manera posible. Muchos errores son específicos de cada dispositivo y el controlador apropiado debe manejarlos, pero el marco de trabajo para el manejo de errores es independiente del dispositivo.

Asignación y liberación de dispositivos dedicados

Es responsabilidad del sistema operativo examinar las peticiones de uso de los dispositivos y aceptarlas o rechazarlas, dependiendo de si el dispositivo solicitado está o no disponible.

Tamaño de bloque independiente del dispositivo

Los distintos discos pueden tener diferentes tamaños de sectores. Es responsabilidad del software independiente del dispositivo ocultar este hecho y proporcionar un tamaño de bloque uniforme a los niveles superiores; por ejemplo, al tratar varios sectores como un solo bloque lógico.

Spooling

El **spooling** (Simultaneous Peripheral Operations On-Line) en sistemas operativos es una técnica utilizada para gestionar el acceso a dispositivos periféricos, como impresoras, discos y otros recursos compartidos. Su objetivo es optimizar el uso de estos dispositivos y permitir que varios procesos accedan a ellos de manera eficiente.

¿Cómo funciona el spooling?

1. **Almacenamiento intermedio en disco:** En lugar de enviar directamente los datos al dispositivo periférico, como una impresora, los datos se almacenan temporalmente en una cola en el disco (a menudo llamada **cola de spool**). Cada tarea, por ejemplo, un documento a imprimir, se coloca en esta cola de espera.
2. **Independencia del proceso:** El proceso que solicitó la operación (por ejemplo, una impresión) no necesita esperar a que el dispositivo esté disponible o termine de procesar la solicitud. Una vez que los datos se han escrito en la cola de spool, el proceso puede continuar con otras tareas, mejorando el rendimiento y la eficiencia.
3. **Procesamiento en segundo plano:** Un **demonio de spool** o un proceso encargado del spooling se ejecuta en segundo plano, tomando los trabajos de la cola de spool y enviándolos al dispositivo periférico a medida que este esté disponible.

Ventajas del spooling

- **Mejor uso de los recursos:** El spooling permite que la CPU siga ejecutando otros procesos mientras el periférico está ocupado, lo que optimiza el uso del tiempo de CPU.
- **Aprovechamiento de dispositivos lentos:** Dispositivos como impresoras suelen ser mucho más lentos que la CPU. El spooling permite a los procesos seguir ejecutándose sin estar limitados por la velocidad de los periféricos.
- **Cola de trabajos:** El spooling permite gestionar múltiples solicitudes de diferentes usuarios o procesos, organizándolas en una cola para su procesamiento secuencial.

Diferencia entre Spooling y Buffering*

- **Spooling** implica el uso del disco como almacenamiento temporal, permitiendo que múltiples trabajos se acumulen y se gestionen de manera ordenada.
- **Buffering** se refiere a un almacenamiento temporal en memoria que ayuda a gestionar pequeñas cantidades de datos mientras se transfieren entre la CPU y el periférico, sin utilizar el disco.

Software de E/S en espacio de usuario

Aunque la mayor parte del software de E/S está dentro del sistema operativo, una pequeña porción de éste consiste en bibliotecas vinculadas entre sí con programas de usuario, e incluso programas enteros que se ejecutan desde el exterior del kernel. Las llamadas al sistema, incluyendo las llamadas al sistema de E/S, se realizan comúnmente mediante procedimientos de biblioteca.

El **software de entrada y salida en el espacio de usuario** es una parte fundamental del manejo de la **entrada/salida (E/S)** en sistemas operativos. En los sistemas modernos, el manejo de E/S se realiza tanto a nivel de **kernel** (núcleo del sistema operativo) como a nivel de **usuario** (espacio de usuario), pero el sistema operativo se encarga de gestionar los detalles de las interacciones directas con el hardware. Ahora vamos a ver cómo se gestiona este proceso:

Funcionamiento del Software de E/S en el Espacio de Usuario

1. *Solicitud, Formato y Encolado de la E/S*

El primer paso comienza cuando un **proceso de usuario** (como una aplicación) solicita realizar una operación de E/S, ya sea leer o escribir datos. Esto se realiza a través de una **llamada al sistema** que transfiere el control al **kernel** del sistema operativo.

El sistema operativo, además de recibir la solicitud, debe **formatear los datos** de manera adecuada para el dispositivo. Por ejemplo, si el dispositivo requiere trabajar en bloques, los datos se organizan en ese formato.

Finalmente, la solicitud de E/S se **pone en una cola** si el dispositivo está ocupado o hay otras solicitudes en espera.

Ejemplo

Si un programa quiere leer un archivo del disco, hace una solicitud, los datos se organizan en bloques y la operación se encola si el disco está ocupado.

2. *Nombramiento, protección, bloqueo, uso de búfer, asignación*

En este paso, se manejan diferentes aspectos esenciales de la operación de E/S para garantizar que la operación se realice de manera segura y eficiente:

- **Nombramiento**: Se asocia el archivo o recurso con un nombre en el sistema de archivos.
- **Protección**: Se verifica que el proceso tiene los permisos adecuados para realizar la operación de E/S.
- **Bloqueo**: Se asegura que el archivo o recurso no esté siendo utilizado por otro proceso.
- **Uso de búfer**: Los datos se almacenan temporalmente en un búfer durante la transferencia.
- **Asignación**: Se asignan recursos, como bloques de memoria, para la operación de E/S.

Ejemplo

Al escribir en un archivo, el sistema verifica permisos, bloquea el recurso para evitar accesos simultáneos y usa un búfer para administrar la transferencia de datos.

3. *Establecer los registros de dispositivo y verificar el estado*

Aquí, el sistema operativo configura el dispositivo de E/S, estableciendo los registros correspondientes para definir qué operación se va a realizar y qué datos se van a transferir. También se verifica el **estado del dispositivo** para asegurarse de que esté disponible para la operación de E/S.

Ejemplo

Antes de escribir en un disco, se configuran los registros del controlador del disco con la dirección de los bloques que se van a escribir, y se verifica que el disco esté listo para la operación.

4. *Despertar el controlador cuando se completa la E/S*

Una vez que el dispositivo ha completado la operación de E/S, genera una **interrupción de hardware** para notificar al sistema operativo. El **manejador de interrupciones** responde, despertando al **controlador de E/S** para que termine el proceso y notifique al proceso de usuario que la operación se ha completado.

Ejemplo

Después de terminar una operación de lectura, el disco genera una interrupción, el manejador de interrupciones se activa y el proceso es notificado para continuar con la ejecución.

5. *Realizar la operación de E/S*

Finalmente, la operación de E/S se lleva a cabo. Esto significa que los datos solicitados se transfieren entre el dispositivo y el sistema. En el caso de lectura, los datos del dispositivo se transfieren a un búfer en la memoria; en el caso de escritura, los datos del búfer se escriben en el dispositivo.

Ejemplo

Al leer un archivo, los datos se transfieren desde el disco al búfer del sistema y luego al proceso que solicitó la lectura.

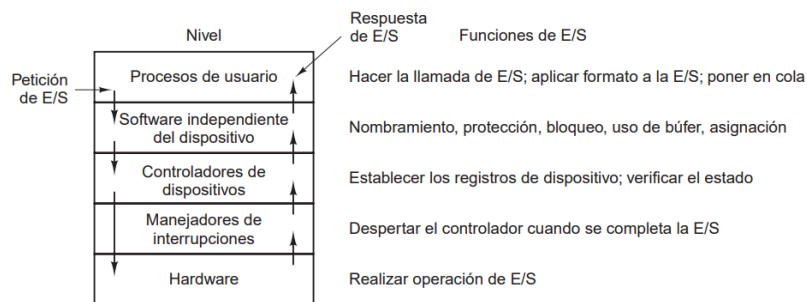


Figura 5-17. Niveles del sistema de E/S y las funciones principales de cada nivel.

Discos

Ahora vamos a estudiar algunos dispositivos de E/S reales. Empezaremos con los discos, que en concepto son simples, pero muy importantes. Los discos son de varios tipos. Los más comunes son los **discos magnéticos** (discos duros y flexibles). Se caracterizan por el hecho de que las operaciones de *lectura y escritura son igual de rápidas*, lo que los hace ideales como memoria secundaria (como paginación o sistemas de archivos, por ejemplo). Algunas veces se utilizan arreglos de estos discos para ofrecer un almacenamiento altamente confiable. Para la distribución de programas, datos y películas, son también importantes varios tipos de **discos ópticos** (CD-ROMs, CD-grabable y DVD).

Geometría física

La **geometría física** de un disco duro define su estructura real: cómo están organizados los datos físicamente en el disco. Los discos duros están formados por **platos** circulares (discos magnéticos) que giran, y los datos se almacenan en **cilindros**, **cabezas** y **sectores**.

- **Cilindro**: Se refiere a un conjunto de pistas que se alinean verticalmente a través de todos los platos.
- **Cabeza**: Cada plato tiene una cabeza de lectura/escritura que accede a los datos en las pistas.
- **Sector**: Es la división más pequeña de la pista, donde se almacenan los datos. Cada sector suele tener un tamaño fijo, por ejemplo, 512 bytes o más en discos modernos.

Geometría virtual

La **geometría virtual** es una representación abstracta que se le presenta al sistema operativo. En lugar de lidiar con la geometría física compleja, se le dice al sistema operativo que el disco tiene, por ejemplo, **x cilindros**, **y cabezas**, y **z sectores por pista**. Esto permite que el sistema operativo trabaje con una estructura más sencilla.

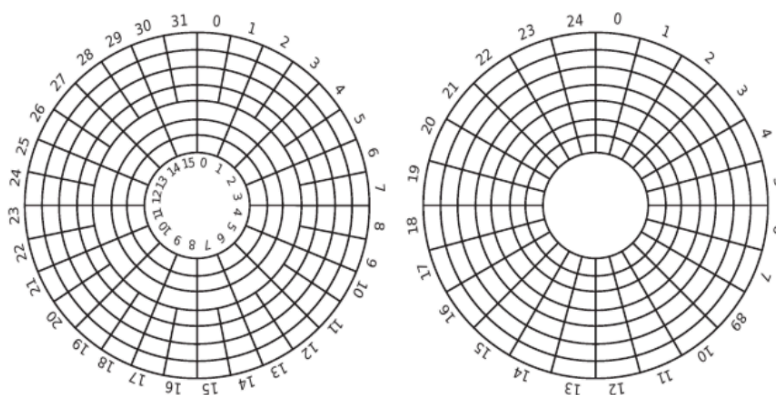
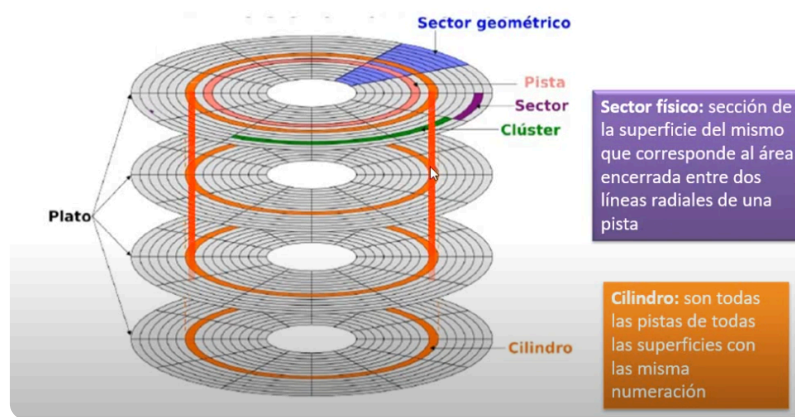


Figura 5-19. (a) Geometría física de un disco con dos zonas. (b) Una posible geometría virtual para este disco.



RAID

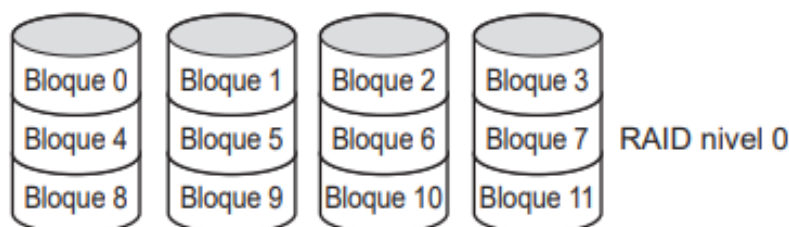
El objetivo de RAID es combinar múltiples discos duros en un **arreglo**, que es gestionado por un **controlador RAID**. Este arreglo aparece como un solo disco para el sistema operativo, aunque en realidad está compuesto por varios discos físicos. La **distribución de los datos** entre los discos permite lograr distintos objetivos, como:

- **Mejorar el rendimiento** (mayor velocidad de lectura/escritura).
- **Aumentar la tolerancia a fallos** (protección contra la pérdida de datos).
- **Mejorar la capacidad de almacenamiento efectiva**.

Controlador RAID: Es un hardware o software que gestiona la distribución de los datos entre los discos y proporciona las funcionalidades de redundancia o mejoras de velocidad, dependiendo del nivel de RAID que se esté utilizando.

RAID 0 (*Striping* - Dividir datos)

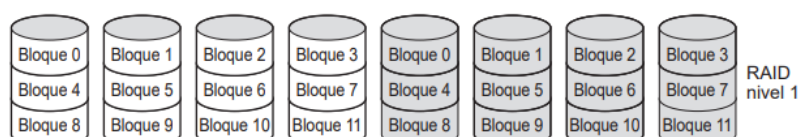
- **Objetivo:** Aumentar el rendimiento (velocidad de lectura/escritura).
- **Funcionamiento:** En RAID 0, los datos se distribuyen (o dividen) entre varios discos. Esto significa que cada archivo se divide en fragmentos, y cada fragmento se almacena en un disco diferente.



Aspecto	Descripción
Ventajas	Las operaciones de lectura y escritura son más rápidas debido al trabajo en paralelo de varios discos. Utiliza el 100% del espacio disponible.
Desventajas	No ofrece tolerancia a fallos; si un disco falla, se pierde toda la información almacenada en el RAID 0.
Uso	Ideal cuando se requiere maximizar el rendimiento y la velocidad, como en edición de video, donde la seguridad de los datos no es prioritaria.

RAID 1 (*Mirroring* - Espejo)

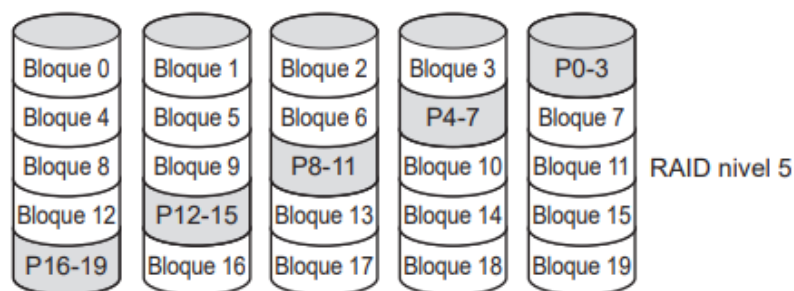
- **Objetivo:** Aumentar la fiabilidad (redundancia de datos).
- **Funcionamiento:** En RAID 1, todos los datos se copian de manera exacta en dos o más discos. Esto significa que si escribes un archivo, se almacena de manera idéntica en todos los discos.



Aspecto	Descripción
Ventajas	No se pierde información si un disco falla, ya que los datos están duplicados. Aumenta la seguridad de los datos y se puede reemplazar un disco sin pérdida.
Desventajas	Sacrifica capacidad, ya que solo se utiliza el espacio del disco más pequeño. Las escrituras son más lentas porque se deben duplicar en ambos discos.
Uso	Ideal para sistemas donde la pérdida de datos es inaceptable, como servidores críticos o almacenamiento de datos personales importantes.

RAID 5 (*Striping* con Paridad)

- **Objetivo:** Equilibrar rendimiento y fiabilidad.
- **Funcionamiento:** RAID 5 utiliza un método de **striping** (como RAID 0) para distribuir los datos entre varios discos, pero también incluye **paridad** para proteger los datos. La paridad es un tipo de información de control que permite reconstruir los datos si uno de los discos falla. Los bloques de paridad no están en un disco fijo, sino que se distribuyen entre todos los discos para mejorar el rendimiento.



Aspecto	Descripción
Ventajas	Si un disco falla, los datos pueden ser reconstruidos utilizando la información de paridad almacenada en los otros discos. RAID 5 es más eficiente en el uso del espacio que RAID 1, ya que utiliza solo el equivalente a un disco para la paridad. Por ejemplo, si tienes tres discos de 1 TB en RAID 5, obtienes 2 TB de almacenamiento utilizable. RAID 5 tiene buen rendimiento en lectura debido a la distribución de los datos, aunque las operaciones de escritura son más lentas que RAID 0 debido al cálculo de la paridad.
Desventajas	La escritura es más lenta debido al cálculo de la paridad. La reconstrucción de datos tras un fallo de disco puede ser lenta.
Uso	Común en servidores y entornos empresariales que requieren una combinación de rendimiento, capacidad y seguridad de datos.